

SHANTO-MARIAM UNIVERSITY OF CREATIVE TECHNOLOGY

Department of CSE & CSIT

Software Requirements Specification (SRS)

FixBuddy – "Your Friendly Local Service Helper"

A Local Handyman Booking Mobile Application

Course Name: Mobile Application and Development

Course Code: CSE 4114

Prepared By,

1. Name: Mahpara Alam Tahia

ID: 223071012

2. Name: Ramim Rashid

ID: 223071043

Semester: 9th

Batch: 31st

Supervised By,

Shad Al Kaisar

Lecturer

Department of CSE & CSIT

1. Introduction

1.1 Purpose

FixBuddy is a cross-platform mobile application designed to solve the problem of finding and booking reliable local service providers in Bangladesh. Currently, users depend on personal word-of-mouth or spend excessive time searching for trusted workers such as plumbers, electricians, carpenters, and tutors.

The purpose of this application is to:

- Provide a centralized digital platform for local service discovery and booking.
- Help everyday users — especially households and individuals — quickly connect with verified local workers.
- Improve trust and transparency through a basic rating and review system.
- Simplify the entire booking workflow with date/time selection and confirmation screens.

1.2 Scope

FixBuddy will be a Flutter-based mobile application targeting Android (primary) and iOS (optional) platforms. The core scope includes:

- A categorized worker directory covering Home Repair (plumbers, electricians, carpenters), Education (tutors), and general local service providers.
- Location-based filtering to find workers by area (e.g., Uttara, Dhanmondi).
- A full booking system with date and time selection and a confirmation screen.
- In-app communication: chat and direct call button.
- Push notifications for booking updates via Firebase Cloud Messaging.
- A basic rating system for workers.

1.3 Objectives

The main objectives of the FixBuddy project are:

- To build a Flutter-based cross-platform mobile application for Android and iOS.
- To implement Firebase authentication for secure user and worker login.
- To create a categorized and searchable worker directory.
- To enable location-based filtering so users can find workers in their area.
- To implement a booking system with date and time selection.
- To provide communication features including in-app chat and a call button.
- To include a basic worker rating system to improve trust.
- To send push notifications for real-time booking status updates.

1.4 Target Users

FixBuddy is designed for three main user groups:

- Households and Individuals (Users): People in Bangladesh who need quick and reliable access to local handymen, repair workers, or tutors without relying on personal networks.

- Service Providers (Workers): Local professionals such as plumbers, electricians, carpenters, and tutors who want a digital presence and a structured way to receive booking requests.
- Platform Administrators (Admin): Personnel responsible for managing the platform, verifying worker profiles, and maintaining service quality.

1.5 Definitions, Acronyms, and Abbreviations

Term	Definition
FixBuddy	The name of the local service booking mobile application.
Flutter	An open-source UI framework by Google for building natively compiled cross-platform apps.
Firebase	Google's Backend-as-a-Service (BaaS) platform used for authentication, real-time database, and push notifications.
Firestore	A cloud-hosted NoSQL database provided by Firebase.
FCM	Firebase Cloud Messaging – a cross-platform messaging solution for push notifications.
FR	Functional Requirement.
NFR	Non-Functional Requirement.
SRS	Software Requirements Specification.
UI/UX	User Interface / User Experience.
API	Application Programming Interface.

2. Overall Description

2.1 Product Perspective

FixBuddy is a standalone mobile application that integrates with Google Firebase as its backend. It does not depend on any existing third-party service booking platform. The app will operate as follows:

- Frontend: Flutter (Dart) application running on Android and iOS devices.
- Backend: Firebase handles all authentication, real-time data, and push notifications.
- Database: Firestore stores user profiles, worker listings, booking records, and ratings.
- Notifications: Firebase Cloud Messaging (FCM) delivers push notifications for booking status updates.

The app will be a self-contained digital ecosystem for local service discovery, booking, and communication in Bangladesh.

2.2 Product Functions

The main functionalities of FixBuddy include:

- User registration and login via email/password through Firebase Authentication.
- Worker profile browsing with categorized listings (plumber, electrician, carpenter, tutor, etc.).
- Location-based filtering to find workers by geographic area.
- Booking system: select service date and time, submit booking requests.
- Booking confirmation screen displaying full booking details.
- In-app chat between users and workers.
- Direct call button for quick contact with a worker.
- Basic rating and review system for workers.
- Push notifications for booking updates and status changes.
- Admin dashboard for managing users, workers, and listings.

2.3 User Classes and Characteristics

User Role	Characteristics	Permissions
User	General public; households and individuals needing local services.	Search workers, filter by area, book services, chat, call, and rate workers.
Worker	Local service professionals such as plumbers, electricians, tutors.	Manage profile, view and respond to booking requests, chat with users.
Admin	Platform manager; technical staff overseeing operations.	Manage all users and workers, approve listings, moderate content, handle disputes.

2.4 Operating Environment

Component	Details
-----------	---------

Platform	Android (primary target), iOS (secondary)
Framework	Flutter (Dart)
Backend	Firebase (Authentication, Firestore, FCM)
IDE	Android Studio / Visual Studio Code
Min Android Version	Android 10 (API Level 29) or above
Language	Dart

2.5 Design and Implementation Constraints

- The application must be built using the Flutter framework.
- Firebase must be used as the backend service for authentication and data storage.
- The app must support Android 10 or above as the minimum OS version.
- The development team consists of two members with a fixed academic deadline.
- The application must function on both Android and iOS from a single codebase (Flutter).
- All UI components must be optimized for mobile screen sizes.

2.6 Assumptions and Dependencies

- Users must have an active internet connection to browse workers, make bookings, and use chat features.
- Firebase servers must be active and accessible for the app to function.
- Users are assumed to register with a valid email address.
- Workers are assumed to keep their availability and profile information updated.
- Push notifications require the device to have Google Play Services (Android) or APNs configured (iOS).
- Location-based filtering assumes users will allow location permissions or manually select their area.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement Description	Priority
FR1	The system shall allow users to register using a valid email address and password.	High
FR2	The system shall allow registered users to log in and log out securely.	High
FR3	The system shall display a categorized list of available workers (plumber, electrician, carpenter, tutor, etc.).	High
FR4	The system shall allow users to filter workers by geographic area (e.g., Uttara, Dhanmondi).	High
FR5	The system shall allow users to view a worker's detailed profile including name, category, area, and rating.	High
FR6	The system shall allow users to select a date and time and submit a booking request to a worker.	High
FR7	The system shall display a booking confirmation screen with full booking details after a request is submitted.	High
FR8	The system shall allow workers to view and respond to incoming booking requests.	High
FR9	The system shall provide an in-app chat feature between users and workers.	Medium
FR10	The system shall provide a call button on the worker's profile for direct contact.	Medium
FR11	The system shall allow users to rate workers after a completed service.	Medium
FR12	The system shall send push notifications to users and workers for booking status updates.	Medium
FR13	The system shall allow workers to manage and update their profile information.	Medium
FR14	The system shall provide an admin panel to manage users, workers, and service listings.	Low

3.2 Non-Functional Requirements

ID	Requirement Description	Type
NFR1	The app should load the worker listing screen within 3 seconds on a standard mobile connection.	Performance
NFR2	The app must support Android 10 and above; iOS 13 and above.	Compatibility

NFR3	All user data and passwords must be encrypted and managed securely through Firebase Authentication.	Security
NFR4	The app UI must be clean, simple, and usable for users with minimal technical experience.	Usability
NFR5	The application must handle up to 500 concurrent users without performance degradation.	Scalability
NFR6	The system must achieve at least 99% uptime, relying on Firebase's cloud infrastructure.	Reliability
NFR7	Push notifications shall be delivered within 5 seconds of a booking status change.	Performance
NFR8	The application codebase must be maintainable and structured following Flutter best practices.	Maintainability

3.3 External Interface Requirements

User Interface:

- The app will feature a clean, mobile-first UI built with Flutter widgets.
- Home screen: categorized worker listings with search and filter options.
- Worker profile screen: photo, name, category, area, rating, chat, and call button.
- Booking screen: calendar date picker, time slot selection, and confirm button.
- Confirmation screen: summary of booking details.
- Notification panel: list of booking updates.

Software Interfaces:

- Firebase Authentication: for user/worker registration and login.
- Cloud Firestore: for storing and retrieving user, worker, and booking data in real time.
- Firebase Cloud Messaging (FCM): for delivering push notifications.
- Flutter Dart packages: for UI components, state management, and API communication.

Hardware Interfaces:

- Android smartphones with Android 10 or above.
- Device microphone and dialer for the call feature.
- Internet connectivity (Wi-Fi or mobile data) is required for full functionality.

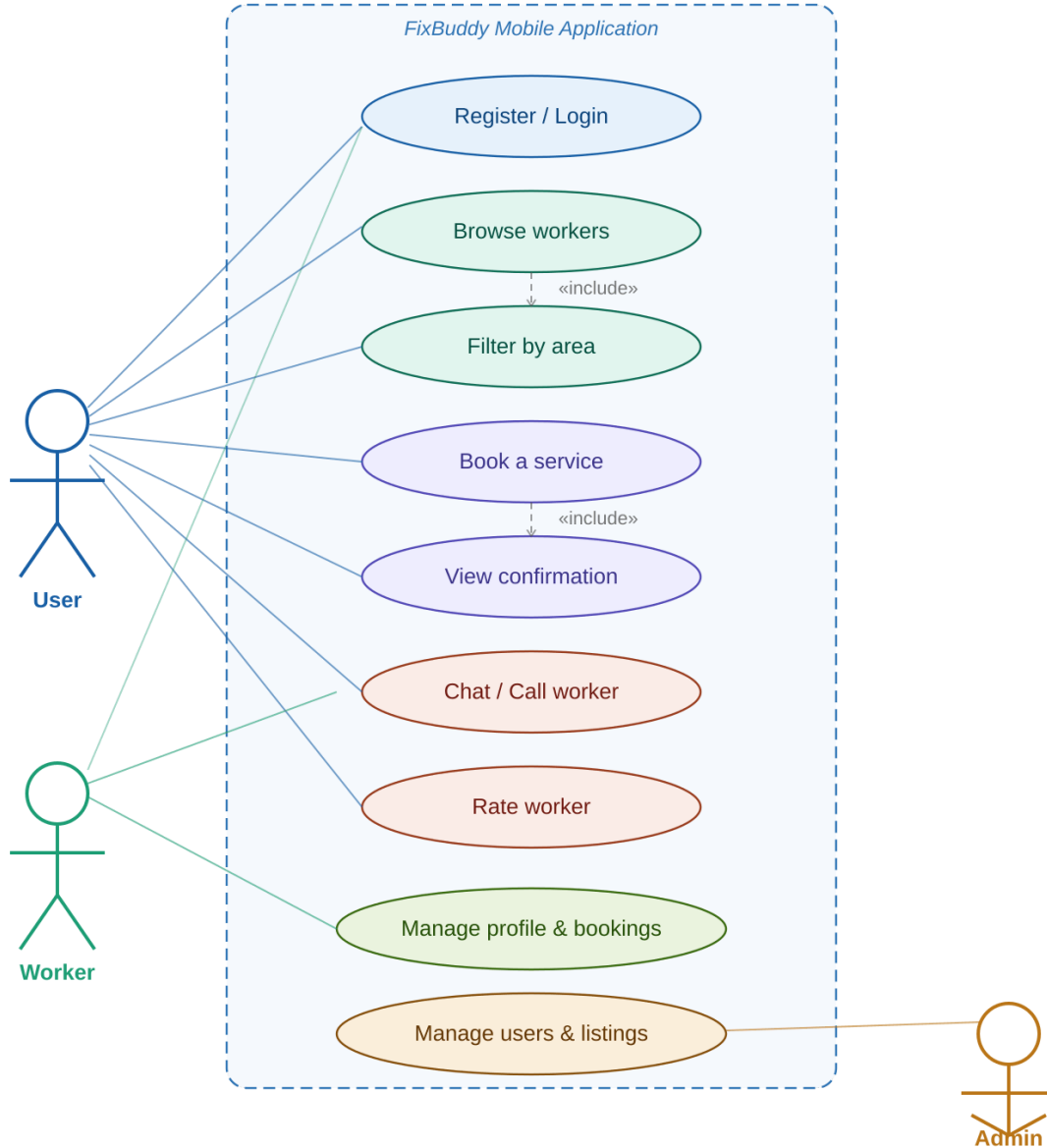
4. System Models

4.1 Use Case Summary

The following table summarizes the major use cases for the FixBuddy application:

Actor	Use Case	Description
User	Register / Login	Create account and securely log in.
User	Browse Workers	View categorized worker listings.
User	Filter by Area	Filter workers by geographic location.
User	Book a Service	Select date/time and submit a booking.
User	Chat / Call Worker	Communicate directly with a worker.
User	Rate Worker	Submit a rating after service completion.
Worker	Manage Profile	Update personal and service information.
Worker	Respond to Bookings	Accept or decline incoming requests.
Admin	Manage Users & Workers	Approve, edit, or remove accounts and listings.

FixBuddy – Use Case Diagram

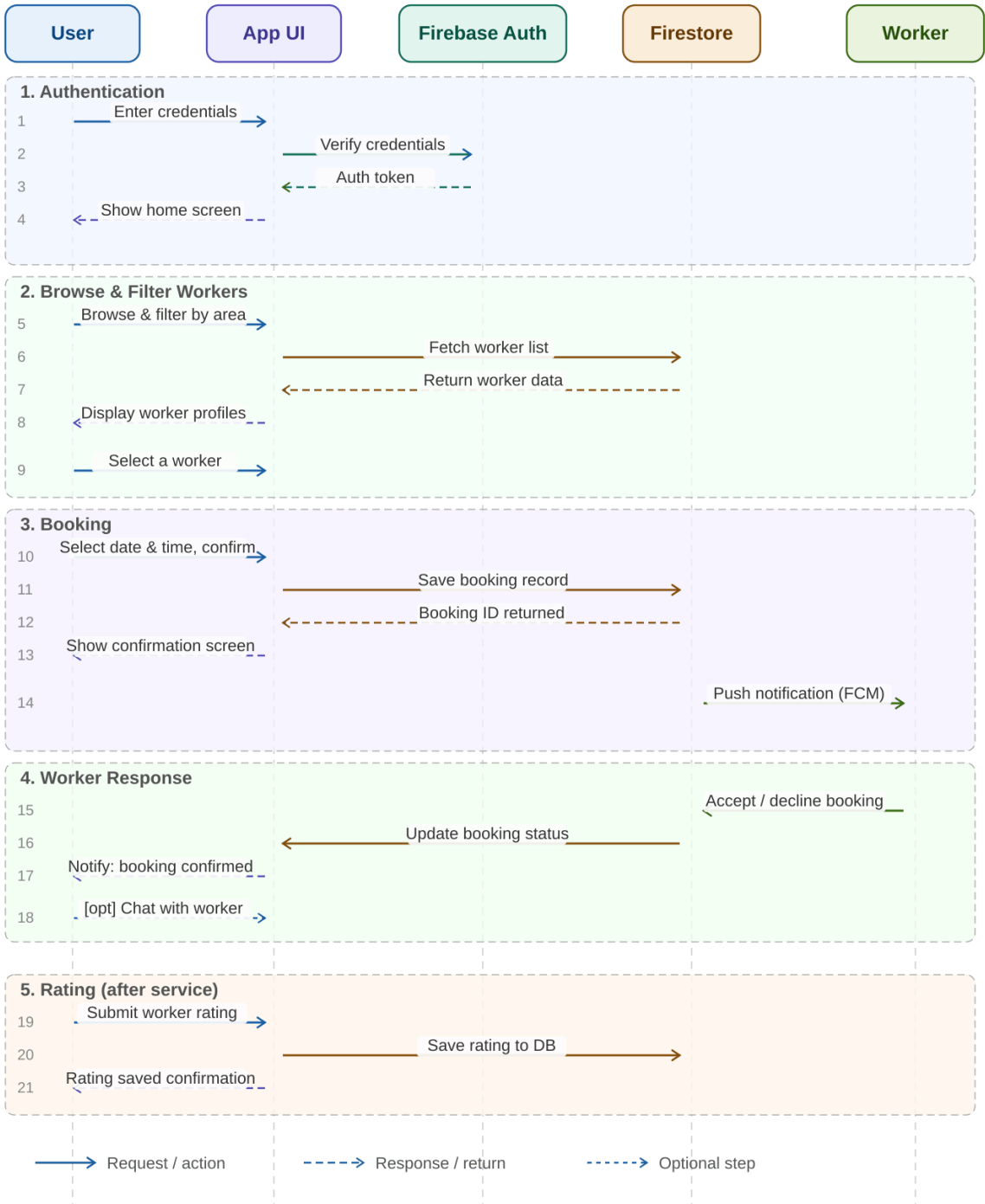


4.2 Sequence Flow – Booking a Service

The sequence below describes the standard booking process in FixBuddy:

- Step 1: User opens the app and logs in via Firebase Authentication.
- Step 2: User browses the worker directory and applies an area filter.
- Step 3: User selects a worker and views their profile details.
- Step 4: User taps 'Book Now', selects a date and time, and submits the booking.
- Step 5: Firestore stores the booking record and sends an FCM notification to the worker.
- Step 6: Worker receives the notification, views the request, and accepts or declines.
- Step 7: User receives an FCM notification with the booking status update.
- Step 8: User views the booking confirmation screen with full details.

FixBuddy – Sequence Diagram (Booking Flow)



5. Project Management

5.1 Development Tools

Tool / Technology	Purpose
Flutter (Dart)	Cross-platform mobile app development framework.
Firebase Authentication	Secure user/worker registration and login.
Cloud Firestore	Real-time NoSQL database for storing app data.
Firebase Cloud Messaging	Push notifications for booking updates.
Android Studio	Primary IDE for Flutter development and Android emulation.
Visual Studio Code	Lightweight code editor with Flutter plugins.
Git / GitHub	Version control and collaborative code management.
Figma	UI/UX design and wireframing tool.

5.2 Development Timeline

Phase	Task	Activities	Duration
1	Planning & Requirement Analysis	SRS writing, use case design	1 Week
2	UI/UX Design	Wireframing in Figma	1 Week
3	Firebase Setup & Authentication	Firebase integration, login/register screens	1 Week
4	Core Feature Development	Worker listing, area filter, booking system	2 Weeks
5	Communication Features	In-app chat, call button integration	1 Week
6	Notifications & Ratings	FCM push notifications, rating system	1 Week
7	Testing & Bug Fixing	Unit testing, UI testing, debugging	1 Week
8	Final Submission & Documentation	Final report, SRS, demo preparation	1 Week

6. Appendix

6.1 References

- Flutter Official Documentation: <https://flutter.dev/docs>
- Firebase Documentation: <https://firebase.google.com/docs>
- Cloud Firestore Guide: <https://firebase.google.com/docs/firestore>
- Firebase Cloud Messaging (FCM): <https://firebase.google.com/docs/cloud-messaging>
- Flutter pub.dev (package repository): <https://pub.dev>
- Android Developer Guidelines: <https://developer.android.com>
- Course Material: Mobile Application and Development, CSE 4114, Shanto-Mariam University of Creative Technology.
- Project Proposal Document: FixBuddy – Your Friendly Local Service Helper (2024).

6.2 Technology Stack Summary

Layer	Technology	Purpose
Frontend	Flutter (Dart)	Cross-platform UI development
Backend	Firebase	Real-time data and services
Database	Cloud Firestore	Store user, worker & booking data
Notifications	Firebase Cloud Messaging	Push notification delivery
Authentication	Firebase Authentication	Secure login and registration